

DANMARKS TEKNISKE UNIVERSITET



MODELING CO₂ CONCENTRATIONS USING HIDDEN
MARKOV MODELS

2026

SPECIAL COURSE IN STATISTICAL MODELLING

Mads Holme Håkonsson
s234936

Supervisor
Jan Klobbenborg Møller

1. juni 2026

AI Declaration

AI completion has been on when writing code and no Agentic AI has been used in the written of the source code. Agentic AI has been used to write LaTeX formatting code and plots in the drivers directory. Agnetic AI has been used to write some of the tests in the tests directory. AI has been used for formatting LaTeX tables, figures and reformatting text in the report. All academic content, including the mathematical equations, h as been written by the author without the use of AI.

Indhold

1	Introduction	3
2	Software	4
2.1	Class overview	4
2.2	Design Principles	4
2.3	Primary Libraries	4
3	General Theory	5
3.1	Forward Algorithm	5
3.2	Forecast Pseudo Residuals	5
3.3	Test Statistics	6
4	Models & Results	7
4.1	Ordinary HMM model	8
4.1.1	Theory	8
4.1.2	Results	9
4.2	AR(1)-HMM	10
4.2.1	Theory	10
4.2.2	Results	10
4.3	AR(2)-HMM	12
4.3.1	Theory	12
4.3.2	Results	12
4.4	AR(1)-HMM(2)	13
4.4.1	Theory	13
4.4.2	Results	13
4.5	AR(2)-HMM(2)	16
4.5.1	Theory	16
4.5.2	Results	16
5	Model Comparison	19
6	Conclusion	20
7	Future Work	21
8	Appendix	23
8.1	Source Code Repository	23
8.2	Source code & Environment	23
8.3	CO ₂ Data	24
8.4	Class Diagrams	25
8.5	Class Design	27

1 Introduction

Hidden Markov Models (HMMs) are a powerful tool for modeling sequential data, where the underlying states of the system are not directly observable. They have been widely used in various fields such as speech recognition, natural language processing, and bioinformatics. (Ultralytics LLC, 2024).

The goal of this project is to inference if a person is in a room based on CO₂ data collected from a sensor in the room. To do this, we will use a Hidden Markov Model to model the relationship between the observed CO₂ levels and the hidden states of whether a person is in the room or not and if a window is open or not. When training the models, we will use **JAX** (Bradbury et al., 2018) and **EQUINOX** (Kidger and Garcia, 2021) for automatic differentiation and optimization, and we will use **Optax** (Hessel et al., 2020) as our optimization library.

In this report, we will implement 5 nested HMM's to best describe the state of the room given the CO₂ data, and we will compare the models using test statistics such as AIC, BIC and Likelihood Ratio Tests.

We find that the best model is the AR(2) second-order HMM, which has both autoregressive emissions and a second-order latent chain and in all cases, the more complex model is favoured over the simpler model.

By looking at the ACF plot of the pseudo residuals, we can see that the AR(2) second-order HMM still doesn't capture all of the temporal structure in the data, which suggests that there is still room for improvement in the model.

This report is written for the purpose of given a concise outline of the projects achivements, design choices, model results and interpretaiton. The report is written for future students to be able to easily carry on the the project and extend it in some way.

2 Software

This section will give an explanation of the classes implemented, the design choices made and the primary libraries used. A repository link can be found in 8.1 and the environment details and short source code overview can be found in 8.2. Class diagrams can be found in 8.4.

2.1 Class overview

The source code is organized into 6 main components: the `HMM` class, the `Emissions` class, the `Transitions` class, the `HMMParams` class, the `Inference` class, and the `Solvers` class. All classes except `HMM` are abstract interfaces. The `HMM` class acts as the main entry point, providing a simple interface for training and inference by delegating to the other components. The key design principle is that concrete implementations of `Emissions` and `Transitions` are injected at runtime, allowing different HMM variants to be used interchangeably. A detailed explanation of each class and the motivation behind the design choices can be found in 8.5, and class diagrams are provided in 8.4.

2.2 Design Principles

An Object-Oriented Programming (OOP) design was chosen for this project to promote modularity, extensibility, and separation of concerns. The Dependency Inversion Principle (DIP) was central to the design of all core components. They are defined as abstract interfaces, so different implementations can be swapped in without affecting the rest of the system. This modularity is also motivated by the mathematical structure of HMMs themselves. The forward algorithm, and HMM inference in general, depends only on two quantities. The emission density $p(y_t | x_t)$ and the transition probability $p(x_t | x_{t-1})$. It is entirely agnostic to how those quantities are computed. By encapsulating each behind an abstract interface, different HMM variants can be used interchangeably without modifying the inference or optimization code, so modularity follows naturally from the structure of the model. This also makes it straightforward to extend the project by adding new emission distributions, transition models, or inference algorithms without modifying existing code.

2.3 Primary Libraries

The primary libraries used in this project are `JAX` (Bradbury et al., 2018), `EQUINOX` (Kidger and Garcia, 2021), and `Optax` (Hessel et al., 2020).

`JAX` was chosen over `NumPy` for its automatic differentiation capabilities and performance optimizations. However, `JAX` is designed for functional programming and does not natively support classes or object-oriented design. Therefore, `EQUINOX` was chosen as it's a library built on top of `JAX` that provides a simple and flexible interface for defining classes as pytrees.¹, `JAX` can then automatically differentiate through the classes and the leaves of the pytree.

`Optax` was chosen as the optimization library for its seamless integration with `JAX` and `EQUINOX`, providing a wide range of optimization algorithms and utilities for training machine learning models. `Optax` was purely chosen as it's the default optimization library for `JAX` and there may be a better choice for this project, but it was not explored due to time constraints.

¹A pytree is a nested data structure in `JAX`: <https://docs.jax.dev/en/latest/pytrees.html>

3 Genel Theory

In this section, we will present the type of forward algorithm used to compute the likelihood and the pseudo residuals used to evaluate the model fit.

3.1 Forward Algorithm

The forward algorithm implemented is called recursive filtering (Møller et al., 2024) as it constructs the current statedistribution $p(X_t | Y_{1:t})$ recursively from the previous state distribution $p(X_t | Y_{1:t-1})$.

Algorithm 1 Recursive Filtering Forward Algorithm

Require: Observations $\mathbf{y} = (y_1, \dots, y_T)$, covariates $\mathbf{x} = (x_1, \dots, x_T)$, initial state distribution $\mathbf{u}_0 \in \mathbb{R}^{1 \times N}$, HMM parameters θ

Ensure: Predicted distributions $\{\mathbf{u}_t\}$, filtered distributions $\{\mathbf{u}_{t|t}\}$, step likelihoods $\{f_t\}$

```

1: for  $t = 1, \dots, T$  do
2:    $\Gamma_t \leftarrow \text{TRANSITIONMATRIX}(\theta, t, y_t, x_t)$  ▷  $N \times N$  transition matrix
3:    $\mathbf{u}_{t|t-1} \leftarrow \mathbf{u}_{t-1|t-1} \cdot \Gamma_t$  ▷ Predicted state distribution,  $1 \times N$ 
4:    $\mathbf{g}_t \leftarrow \text{DENSITY}(\theta, t, y_t, x_t)$  ▷ Emission densities for all states,  $1 \times N$ 
5:    $f_t \leftarrow \sum_{i=1}^N [\mathbf{u}_t \odot \mathbf{g}_t]_i$  ▷ Marginal likelihood at time  $t$ 
6:    $f_t \leftarrow \max(f_t, 10^{-10})$  ▷ Clip to avoid division by zero
7:    $\mathbf{u}_{t|t} \leftarrow \frac{\mathbf{u}_t \odot \mathbf{g}_t}{f_t}$  ▷ Filtered state distribution (normalized)
8: end for
9: return  $\{\mathbf{u}_t\}, \{\mathbf{u}_{t|t}\}, \{f_t\}$ 

```

where $\odot$ denotes element-wise multiplication of vectos.

The log-likelihood is obtained by summing the log of the step likelihoods:

$$\mathcal{L}(\theta) = \sum_{t=1}^T \log f_t \quad (1)$$

3.2 Forecast Pseudo Residuals

A method to access the goodness-of-fit of the HMM is to compute the pseudo residuals. For a introduction to pseudo residuals, see chapter 6.2 in (Zucchini et al., 2016).

The Forecast Pseudo Residuals used is the standard normal inverse of the cummulative distribution function (CDF) of the one-step-ahead predictive distribution. If the model is is good, the pseudo residuals should be approximately standard normally distributed.

Briefly desribed, a forecast pseudo residual for a HMM is defined as

$$z_{t+1} = \Phi^{-1}(F_Y(y_{t+1} | Y_{1:t})) \quad (2)$$

where Φ^{-1} is the inverse CDF of the standard normal distribution and F_Y is the predictive CDF of y_{t+1} given $Y_{1:t}$, defined as

$$F_Y(y_{t+1} | Y_{1:t}) = \sum_{i=1}^N u_{t+1|t}^i F_i(y_{t+1} | X_t = i) \quad (3)$$

where X_t is the hidden state.

Because z_{t+1} now should follow a standard normal distribution, we can use normality tests to assess the goodness-of-fit of the model.

In this project, we use the QQ-plot to visually assess the normality of the pseudo residuals.

3.3 Test Statistics

Other methods to assess the goodness of fit of the HMM model include the AIC (Wikipedia contributors, 2026a) and BIC (Wikipedia contributors, 2026b) test statistics. These are defined as

$$\text{AIC} = 2k - 2\mathcal{L} \quad (4)$$

$$\text{BIC} = k \log(n) - 2\mathcal{L} \quad (5)$$

where k is the number of parameters in the model, n is the number of observations and $\mathcal{L}$ is the log-likelihood of the model.

The AIC and BIC test statistics are used to compare different models, where a lower value initiates a better fit.

However, a lot of the models implemented are nested models, so a Likelihood Ratio Test (LRT) (Wikipedia contributors, 2026c) can also be used to compare the models. The LRT test statistic is defined as

$$\text{LRT} = 2(\mathcal{L}_1 - \mathcal{L}_0) \quad (6)$$

where $\mathcal{L}_1$ is the log-likelihood of the more complex model and $\mathcal{L}_0$ is the log-likelihood of the simpler model.

The LRT test statistic follows a chi-squared distribution with degrees of freedom equal to the difference in the number of parameters between the two models.

4 Models & Results

For general theory on HMM's, see (Zucchini et al., 2016) and (Møller et al., 2024) chapter 11. All the models implemented are 4 state Gaussian HMMs (emission density is a Gaussian distribution) and we have kept the the mean value of state 1 fixed at 400 as this is the baseline CO₂ level in the room when no one is present and all windows are closed. This is also shown in the figure 7.

Five models are considered, organised along two axes: the order of the AR component in the emission and the order of the latent Markov chain. Their relationship is shown in Figure 1.

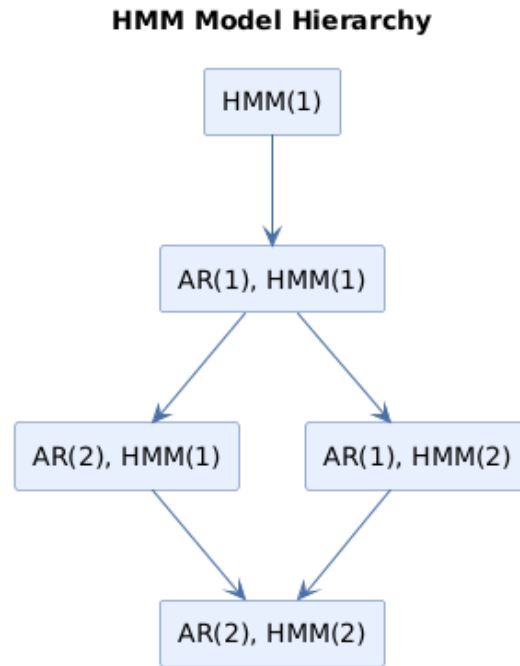


Figure 1: Hierarchy of the HMM models considered in this section. Each extension adds either an autoregressive emission component (AR order) or a higher-order latent chain (HMM order), culminating in the AR(2) second-order HMM.

For each level in the hierarchy tree in 1 starting from HMM(1), a method for capturing the temporal structure of the data is added. The HMM(1) is the ordinary HMM, which does not capture any temporal structure in the data.

4.1 Ordinary HMM model

4.1.1 Theory

The ordinary HMM model is the simplest model implemented in this project and the one most commonly used in the literature. The transition probabilities are constant and do not depend on time, the observations or any covariates, and so does the emission density.

For this model, we want to estimate the parameters $\theta = \{\Gamma, \mu, \sigma\}$ where $\Gamma \in \mathbb{R}^{n \times n}$ is the transition matrix, $\mu \in \mathbb{R}^n$ is the mean vector of the Gaussian emission density and $\sigma \in \mathbb{R}^n$ is the variance vector of the Gaussian emission density. Because the markov chain is finite, discrete and homogeneous, a unique strictly positive stationary distribution δ exists (chapter 1.3.2 (Zucchini et al., 2016)) and is given by

$$\delta = \mathbf{1}^T (I - \Gamma + E)^{-1} \quad (7)$$

where E is a matrix of ones.

To handle the natural bounds on the standard deviation parameters and transition probabilities, the parameters were reparameterized as follows:

$$\sigma_i = \exp(\eta_i) \quad (8)$$

For the transition matrix, the $n(n-1)$ off-diagonal entries are parametrized. The diagonal is used as the reference category with log-odds fixed to zero such that a transition matrix on the from

$$\xi = \begin{bmatrix} - & \xi_{12} & \cdots & \xi_{1n} \\ \xi_{21} & - & \cdots & \xi_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \xi_{n1} & \xi_{n2} & \cdots & - \end{bmatrix} \quad (9)$$

Concretely, the unconstrained parameters are stored as a matrix $\xi \in \mathbb{R}^{n \times (n-1)}$ that holds the off-diagonal logits. The transition matrix is then obtained by exponentiating these logits in the off-diagonal positions of an identity matrix (so the diagonal is $\exp(0) = 1$) and row-normalizing:

$$\Gamma_{ij} = \begin{cases} \frac{\exp(\xi_{ij})}{1 + \sum_{k \neq i} \exp(\xi_{ik})}, & j \neq i, \\ \frac{1}{1 + \sum_{k \neq i} \exp(\xi_{ik})}, & j = i. \end{cases} \quad (10)$$

The unconstrained parameters $\eta \in \mathbb{R}^n$ and $\xi \in \mathbb{R}^{n \times (n-1)}$ are the quantities that are optimized during the fitting process.

The μ parameters are not bounded, but to avoid label switching for the gaussian distributions, the mean parameters are ordered as $\mu_1 < \mu_2 < \mu_3 < \mu_4$ by the reparameterization

$$\begin{aligned} \mu_1 &= \zeta_1 \\ \mu_i &= \zeta_1 + \sum_{k=2}^i \exp(\zeta_k) \end{aligned} \quad (11)$$

where $\zeta \in \mathbb{R}^n$ is the unconstrained parameter vector that is optimized during the fitting process.

4.1.2 Results

The model was fitted on the CO₂ observations by minimising the negative log-likelihood with L-BFGS, with μ_1 frozen at 400 ppm as discussed above. The estimated transition matrix is shown in Table 1 and the estimated emission parameters in Table 2.

	State 1	State 2	State 3	State 4
State 1	0.8496	0.1476	0.0029	0.0000
State 2	0.0828	0.8486	0.0686	0.0000
State 3	0.0004	0.1605	0.6841	0.1550
State 4	0.0000	0.0000	0.0718	0.9282

Tabel 1: Estimated transition matrix for the ordinary 4-state HMM.

The transition matrix has a strongly dominant diagonal with probabilities between 0.68 and 0.93. Some transition is not possible in the matrix as to go from state 1 or 2 to state 4 or from state 4 to state 1 or 2. We see that for each state, there is a least 0.8 probability of switching state and this tells us that the model doesn't get stuck in 1 state for too long or forever.

State	Mean ($\hat{\mu}$)	Std. Dev. ($\hat{\sigma}$)
State 1	400.000	21.748
State 2	535.684	76.573
State 3	889.928	110.981
State 4	1306.484	197.410

Tabel 2: Estimated emission parameters (Gaussian) for the ordinary 4-state HMM.

The four estimated state means in Table 2 (400, 536, 890, 1306 ppm) admit a natural interpretation as a *baseline*, *low*, *medium* and *high* CO₂ levels. The emission standard deviations grow with the mean, which makes sense as the we would expect more volatility in the CO₂ signal when people are present and closing and opening windows.

From these two tables, a interpretation of the states can be that

- state 1 is the baseline state where occupants has not entered the room in a long time.
- state 4 is the when the occupants has been in the room for a long time and the CO₂ level is very high.
- state 2 and 3 are intermediate states where the occupants has entered the room but not for a long time or windows has been opnend before reaching state 4, so the CO₂ level is higher than the baseline but not as high as in state 4.

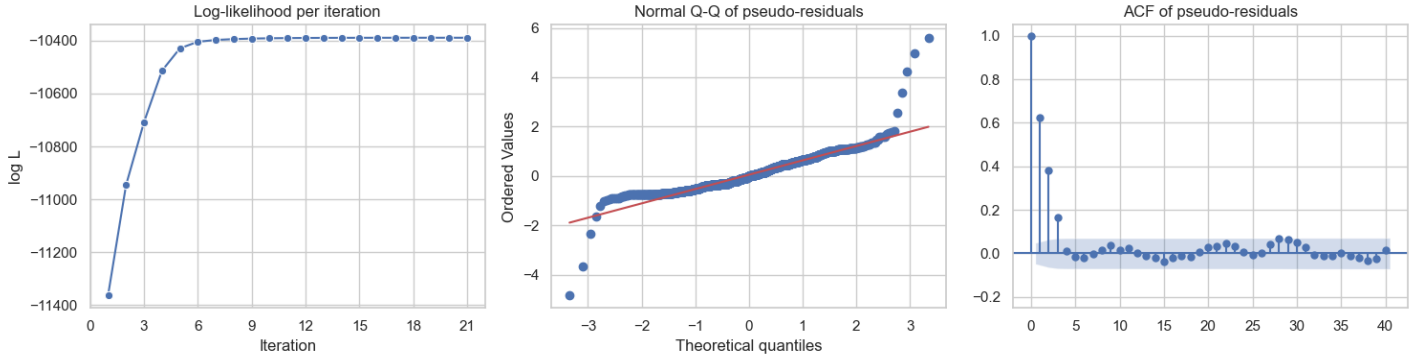


Figure 2: Diagnostic plots for the ordinary 4-state HMM. Left: log-likelihood per iteration. Centre: normal Q-Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

Figure 2 summarises the fit. The log-likelihood trace (left panel) shows that the optimiser converges quickly towards the minimum of the negative log-likelihood. Almost all of the improvement happens in the first few iterations and the optimizer only keeps going due to relative tolerance in the negative log-likelihood is set to $\epsilon = 10^{-6}$. The Q-Q plot (centre) shows that the pseudo-residuals are approximately normal in the middle with some deviation in the tails. The ACF plot however clearly shows that there is autocorrelation in the pseudo-residuals as the 3 first lags are decaying slowly. This indicates that we need to introduce temporal structure between the observations to capture the dynamics of the data better. This motivates the introduction of the AR-HMM in the next section, which adds an autoregressive component to the emission density.

4.2 AR(1)-HMM

4.2.1 Theory

The Autoregressive HMM (AR(1)-HMM) extends the ordinary HMM by letting the emission distribution depend on the previous observation. Conditional on being in state i at time t , the observation is modelled as

$$Y_t \mid X_t = i, Y_{t-1} = y_{t-1} \sim \mathcal{N}(\mu_i + \phi_i(y_{t-1} - \mu_i), \sigma_i^2), \quad (12)$$

Therefore, each states has its own AR(1) coefficient $\phi_i \in (-1, 1)$ in addition to a mean and variance. This may be enough to capture the temporal correlation of the CO₂ signal that the ordinary HMM left in the pseudo-residuals. The transition matrix Γ is parametrised exactly as in the ordinary HMM, and the AR coefficients are reparametrised as $\phi_i = 2\sigma(\psi_i) - 1$, where $\sigma(z) = (1 + e^{-z})^{-1}$ is the logistic sigmoid, to enforce the stationarity constraint $|\phi_i| < 1$.

The unconstrained parameter $\psi_i \in \mathbb{R}$ is the one passed to the optimiser, with inverse mapping $\psi_i = \text{logit}\left(\frac{\phi_i + 1}{2}\right)$.

4.2.2 Results

The ordinary-HMM estimates were used as a warm start for Γ , μ and σ , and the AR coefficients were initialised at $\phi_i = 0.2$ for all states. The estimated transition matrix is shown in Table 3 and the estimated emission parameters in Table 4.

The transition matrix strongly dominant in the diagonal, with probabilities between 0.74 and 0.97. The off-diagonal pattern is similar to the ordinary HMM, but now only 1 transition is

	State 1	State 2	State 3	State 4
State 1	0.7521	0.1685	0.0016	0.0779
State 2	0.2698	0.5960	0.0633	0.0709
State 3	0.0516	0.1027	0.8457	0.0000
State 4	0.0067	0.0492	0.0153	0.9289

Tabel 3: Estimated transition matrix for the AR-HMM.

impossible, which is state 3 to state 4. We switched self transition probabilities between state 2 and 3, meaning AR(1)-HMM says it's more likely to stay in state 2 than state 3, while the ordinary HMM says the opposite.

State	Mean ($\hat{\mu}$)	Std. Dev. ($\hat{\sigma}$)	AR coeff. ($\hat{\phi}$)
State 1	400.000	12.501	0.6235
State 2	509.326	77.704	0.3659
State 3	560.285	11.006	0.9462
State 4	1366.661	102.836	0.8318

Tabel 4: Estimated emission parameters for the AR-HMM (Gaussian with AR(1) residuals).

The estimated state means in Table 4 are similar to the ordinary HMM, with a clear ordering from low to high. We see that state 4 mean is close the HMM state 4 mean, but state 2 and 3 means are closer together than in the ordinary HMM. This validates the interpretation of these two states being transitional states between the baseline and the high-CO₂ state.

All four AR coefficients are positive and substantial, ranging from $\hat{\phi}_2 = 0.37$ up to $\hat{\phi}_3 = 0.95$; the high-CO₂ state (state 4) also has a strong AR component ($\hat{\phi}_4 = 0.83$). This confirms that a non-trivial amount of the temporal structure in the observations is captured by the AR(1) component of the emission density. The positive AR coefficients also make sense as we would expect the CO₂ signal at a timestep t to be positively correlated with the CO₂ ppm at time $t - 1$ by logical thinking.

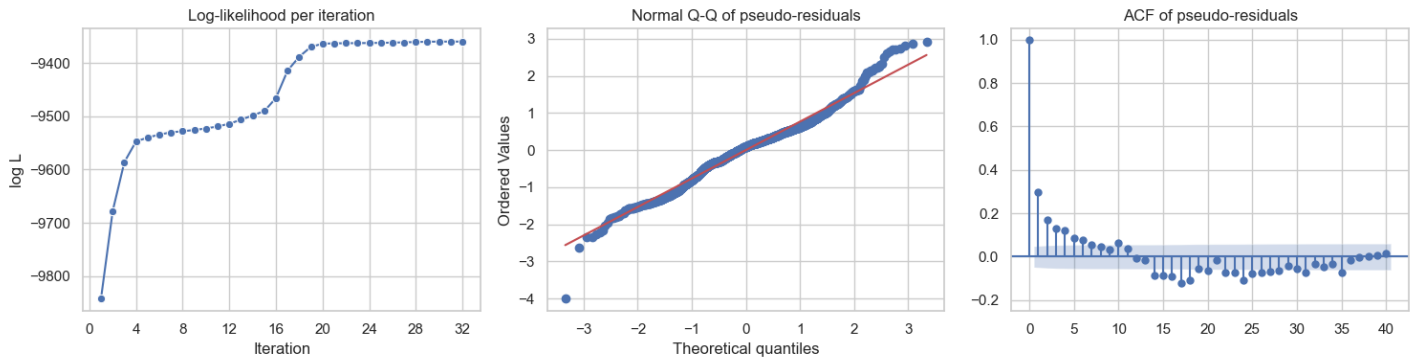


Figure 3: Diagnostic plots for the AR-HMM. Left: log-likelihood per iteration. Centre: normal Q-Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

Figure 3 shows the diagnostics for the AR-HMM. The log-likelihood trace (left panel) shows that the optimiser converges after a few more iterations than for the ordinary HMM, which is expected as the AR-HMM has more parameters. The Q-Q plot of the pseudo-residuals (centre) shows a clear improvement in the normality of the residuals compared to the ordinary HMM,

with only a single outlier in the lower tail and few more in the upper tail. The ACF however seems to have been worsened as there are more lags with significant residual spikes, and the first lags drops quickly to around 0.3 and then decays slowly. We also see that after lag 15, almost all of the lags are negative. This indicates that while the AR(1) component captures a lot of the temporal structure in the data, there is still some autocorrelation left in the residuals. This motivates the introduction of the AR(2)-HMM in the next section, which adds a second-order autoregressive component to the emission density.

4.3 AR(2)-HMM

4.3.1 Theory

The Autoregressive of order 2 HMM (AR(2)-HMM) extends the AR-HMM by letting the autoregressive component depend on the two most recent observations rather than just one. Conditional on being in state i at time t , the observation is modelled as

$$Y_t \mid X_t = i, y_{t-1}, y_{t-2} \sim \mathcal{N}(\mu_i + \phi_{i,1}(y_{t-1} - \mu_i) + \phi_{i,2}(y_{t-2} - \mu_i), \sigma_i^2), \quad (13)$$

so each state now has a pair of AR coefficients $(\phi_{i,1}, \phi_{i,2})$ in addition to a mean and variance.

4.3.2 Results

The AR-HMM estimates were used as a warm start, and the second-order AR coefficients were initialised at $\phi_{i,2} = 0.2$ for all states. The estimated transition matrix is shown in Table 5 and the estimated emission parameters in Table 6.

	State 1	State 2	State 3	State 4
State 1	0.7115	0.2065	0.0000	0.0820
State 2	0.2848	0.5479	0.1055	0.0618
State 3	0.0216	0.1325	0.8459	0.0000
State 4	0.0061	0.0453	0.0171	0.9315

Table 5: Estimated transition matrix for the AR(2)-HMM.

The transition matrix is very close to the one obtained for the AR-HMM. Adding a second lag therefore does not noticeably change the between-state dynamics of the model.

State	Mean ($\hat{\mu}$)	Std. Dev. ($\hat{\sigma}$)	AR coeff. ($\hat{\phi}_1$)	AR coeff. ($\hat{\phi}_2$)	$\hat{\phi}_1 + \hat{\phi}_2$
State 1	400.000	10.765	0.7027	-0.0781	0.6247
State 2	525.630	72.083	0.5941	-0.2109	0.3832
State 3	716.015	16.158	0.9596	0.0565	1.0161
State 4	1299.660	97.603	0.9997	-0.2093	0.7904

Table 6: Estimated emission parameters for the AR(2)-HMM (Gaussian with AR(2) residuals).

The first-order AR coefficients in Table 6 remain strongly positive across all states (0.59–1.00), while the second-order coefficients are small and mostly negative, ranging from -0.21 to 0.06. The sum of the AR coefficients resemble the AR(1) coefficients of the previous model, but the first lag is much higher and the second lag is negative, with except for state 3. This tells us that the observation at time t is more positively correlated with the observation at time $t - 1$ and that the observation at time $t - 2$ has a small negative effect on the observation at time t .

Figure 4 shows the diagnostics for the AR(2)-HMM. The log-likelihood converges more quickly to its optimum than AR(1)-HMM, even though we have introduced an additional parameter,

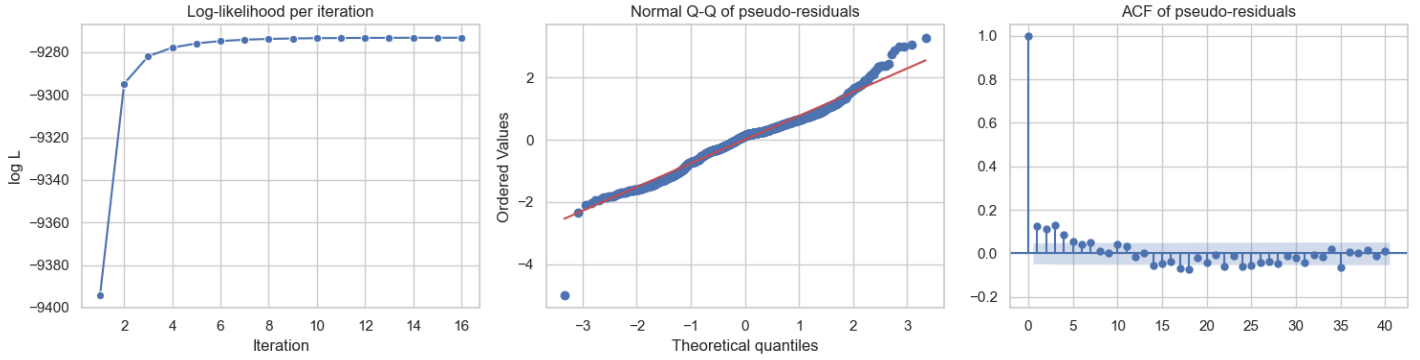


Figure 4: Diagnostic plots for the AR(2)-HMM. Left: log-likelihood per iteration. Centre: normal Q-Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

and the Q-Q plot of the pseudo-residuals is similar to that of the AR(1)-HMM. The ACF plot however shows a clear improvement in the autocorrelation of the residuals, as fewer lags are outside the confidence band and the first lags are smaller than in the AR(1)-HMM. However, the pattern of the ACF is similar to the AR(1)-HMM and there seem to be a pattern we haven't captured yet.

This motivates the introduction of the second-order AR(1)-HMM(2) in the next section, which makes introduces a second order markoc chain.

4.4 AR(1)-HMM(2)

4.4.1 Theory

The Autoregressive second-order HMM (AR(1)-HMM(2)) reformulates the Markov assumption on the latent chain. Now, the next state depends on the two most recent latent states rather than just one. The transition probabilities are therefore indexed by a pair of states,

$$\Gamma_{(i,j),k} = P(X_{t+1} = k \mid X_{t-1} = i, X_t = j), \quad (14)$$

which gives an $n^2 \times n$ tensor of transition probabilities (still row-normalised in k). The emission density is again Gaussian, with the mean depending only on the current state $\mu(x_t)$, but the standard deviation allowed to depend on both the previous and the current state $\sigma(x_{t-1}, x_t)$. When implementing the second-order chain, the state space is augmented to pairs $S_k = (x_{i,t-1}, x_{j,t})$ where $x_{t-k,i}$ is the state i at time $t - k$. $S \in \mathbb{R}^M$ is recognized as the state distribution. We can then define the possible transitions in the augmented transition matrix $\Gamma^* \in \mathbb{R}^{M \times M}$ as

$$\Gamma_{(i,j),(j,k)}^* = \Gamma_{(i,j),k}^* \quad (15)$$

where k is the next state given the current state (i, j) . This is implemented in the code by checking that the current state j is the previous state in the next state. We only store the $n^2 \times n$ log transition probability tensor ξ and not the full $M \times M$ transition matrix Γ^* . When constructing the full transition matrix Γ^* , we fill the entries according to 15 and set the rest of the entries to a high negative value. We then apply the row-wise softmax to get a valid transition matrix.

4.4.2 Results

The estimated transition tensor is shown in Table 7 and the estimated emission parameters in Table 8.

From (s_{t-1}, s_t)	$s_{t+1} = 1$	$s_{t+1} = 2$	$s_{t+1} = 3$	$s_{t+1} = 4$
(1, 1)	0.7633	0.1270	0.0208	0.0889
(1, 2)	0.0705	0.9294	—	—
(1, 3)	0.7629	—	0.0001	0.2370
(1, 4)	0.7065	0.0663	0.1706	0.0565
(2, 1)	0.6903	0.0497	—	0.2600
(2, 2)	0.1675	0.6206	0.0153	0.1966
(2, 3)	—	0.0738	—	0.9262
(2, 4)	0.0242	0.1217	—	0.8542
(3, 1)	0.1370	—	0.8630	—
(3, 2)	0.5383	0.3560	0.1056	0.0001
(3, 3)	0.0425	—	0.0010	0.9564
(3, 4)	0.5295	0.1390	0.3313	0.0001
(4, 1)	0.1148	—	0.3133	0.5719
(4, 2)	0.0124	0.0767	0.3351	0.5757
(4, 3)	—	0.9998	—	0.0002
(4, 4)	—	0.1688	0.0135	0.8177

Tabel 7: Estimated transition probabilities for the AR(1) second-order HMM. Each row is a state pair (s_{t-1}, s_t) ; columns give the next-state probability $P(s_{t+1} | s_{t-1}, s_t)$. A dash (—) indicates zero or negligible probability.

We see in the estimated transition matrix that the previous state you were in effect the transition probabilities of the next state. For example, if you are in state 1 and you were in state 1 in the previous time step, then you have a high probability of staying in state 1 and a low probability of transitioning to state 4. Where at, if you previously were in state 4 and is now in state 1, there is a much higher probability of transitioning to state 4. We also see that state 3 has low self-transition probabilities and the system in general quickly transitions out of state 3, which is in line of state 3 being a transition state between the baseline and the high-CO₂ state. State 2 on the other hand has a higher self-transition probability, and its in general more likely to transition to state 2 than state 3 given previous states. This indicates that state 2 is more of a "sticky" state than state 3.

s_{t-1}	s_t	$\hat{\mu}$	$\hat{\sigma}$	$\hat{\phi}$
1	1	400.000	10.009	0.6797
2	1	400.000	12.472	0.2649
3	1	400.000	5.819	0.9871
4	1	400.000	81.272	1.0000
1	2	446.913	30.971	-0.0550
2	2	446.913	21.726	0.3599
3	2	446.913	32.836	0.1635
4	2	446.913	122.782	0.8514
1	3	574.844	12.245	0.9592
2	3	574.844	114.090	0.6235
3	3	574.844	0.001	0.8921
4	3	574.844	69.077	0.3838
1	4	1461.574	119.463	0.9186
2	4	1461.574	86.911	0.8508
3	4	1461.574	67.035	1.0000
4	4	1461.574	54.907	0.7415

Tabel 8: Estimated emission parameters for the AR(1) second-order HMM. Each row is a state pair (s_{t-1}, s_t) ; $\hat{\mu}$ and $\hat{\sigma}$ are the Gaussian mean and standard deviation of the residual and $\hat{\phi}$ is the AR(1) coefficient.

We see in Table 8 that the state means are similar to the previous models. The means are more evenly spread out and the high CO₂ state (state 4) has a higher mean than in the previous models. The standard deviations is about the same and almost all of the $|\phi|$ vals are below 1. However, two are not. This is for the state pairs (3, 4) and (4, 1). This could be explained by that when moving from state 3 to state 4, the CO₂ level is rising and the mean of the next observation is higher than the current one, which could lead to a high positive AR coefficient. When moving from state 4 to state 1, the CO₂ level is dropping and one would expect a high negative AR coefficient. But looking at the transition matrix 7, we see that its likely to transtion back into a higher CO₂ state after transitioning from state 4 to state 1, which could explain the high positive AR coefficient.

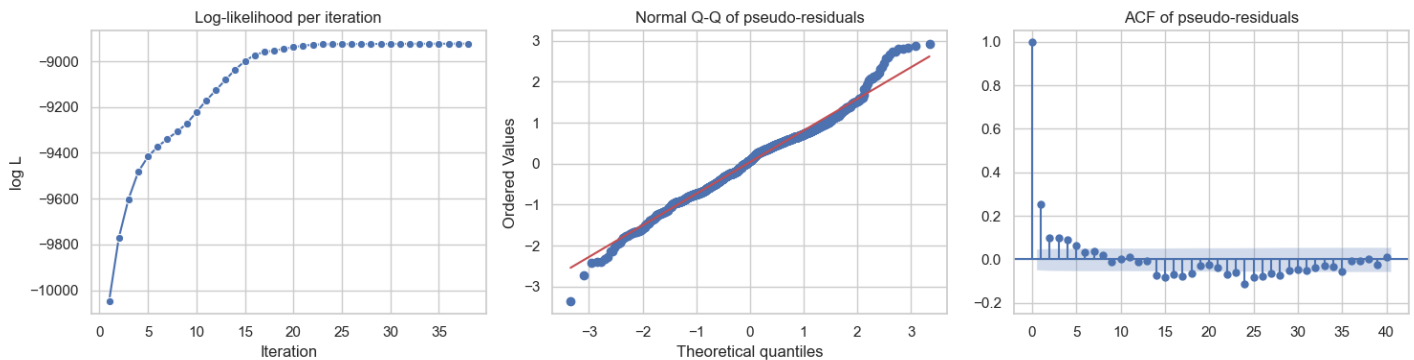


Figure 5: Diagnostic plots for the second-order HMM. Left: log-likelihood per iteration. Centre: normal Q-Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

Figure 5 shows the diagnostics for the second-order HMM. The log-likelihood converges quickly toward the minimum of the negative log-likelihood, in the same way as for the previous models.

The Q-Q plot of the pseudo-residuals is comparable to the AR(2) HMM, and the ACF of the residuals still shows a slow decay with several lags well outside the confidence band, and the diagnostic plots are overall similar to the AR(2)-HMM.

This indicates the the second order chain performs as well as the AR(2) when capturing the temporal structure in the data. This motivates the introduction of the AR(2) second-order HMM in the next section,

4.5 AR(2)-HMM(2)

4.5.1 Theory

The Autoregressive lag 2 Second-order HMM (AR(2)-HMM(2)) combines the two previous extensions. The markov chain is second-order (transitions depend on the pair (x_{t-1}, x_t)) and the emission density carries an AR(2) component whose parameters also depend on the pair (x_{t-1}, x_t) .

4.5.2 Results

Table 9 and the estimated emission parameters in Table 10.

From (s_{t-1}, s_t)	$s_{t+1} = 1$	$s_{t+1} = 2$	$s_{t+1} = 3$	$s_{t+1} = 4$
(1, 1)	0.7192	0.2242	—	0.0565
(1, 2)	0.2137	0.4312	0.3548	0.0002
(1, 3)	0.7218	—	—	0.2782
(1, 4)	0.8343	0.1020	—	0.0637
(2, 1)	0.6948	0.3049	—	0.0003
(2, 2)	0.1289	0.6793	0.0328	0.1590
(2, 3)	—	0.5243	—	0.4757
(2, 4)	—	0.1968	—	0.8032
(3, 1)	0.1582	—	0.8418	—
(3, 2)	0.3295	0.5295	0.1410	—
(3, 3)	0.0008	—	0.3577	0.6415
(3, 4)	0.0409	0.1158	0.8432	0.0001
(4, 1)	0.0620	—	0.3651	0.5729
(4, 2)	—	0.0154	0.3636	0.6210
(4, 3)	0.1742	0.2723	—	0.5534
(4, 4)	—	0.1599	0.0272	0.8129

Table 9: Estimated transition probabilities for the AR(2) second-order HMM. Each row is a state pair (s_{t-1}, s_t) ; columns give the next-state probability $P(s_{t+1} | s_{t-1}, s_t)$. A dash (—) indicates zero or negligible probability.

The transition tensor follows the same overall pattern as the AR(1)-HMM(2). rows like (1, 1), (2, 2) and (4, 4) keep most of their mass on the current state. However, we see that for state 3 that self transition probability are actually lower than the transition probability of transitioning to state 4 from the state pair (3, 3). This pushes state 3 even more into being a transition state between the baseline and the high-CO₂ state.

s_{t-1}	s_t	$\hat{\mu}$	$\hat{\sigma}$	$\hat{\phi}_1$	$\hat{\phi}_2$	$\hat{\phi}_1 + \hat{\phi}_2$
1	1	400.000	7.818	0.7825	-0.0960	0.6865
2	1	400.000	12.809	0.4320	-0.1232	0.3088
3	1	400.000	6.407	1.0000	-0.0030	0.9970
4	1	400.000	126.469	1.0000	0.0524	1.0524
1	2	437.521	38.283	-0.2797	0.1697	-0.1100
2	2	437.521	16.359	0.5665	-0.0767	0.4897
3	2	437.521	67.879	0.7221	-0.1645	0.5576
4	2	437.521	113.350	1.0000	-0.1702	0.8298
1	3	570.631	6.012	0.9285	-0.0006	0.9279
2	3	570.631	98.402	-0.1690	0.4549	0.2859
3	3	570.631	0.000	0.9594	0.0327	0.9922
4	3	570.631	31.589	0.9997	0.0469	1.0466
1	4	1436.337	174.176	1.0000	-0.1229	0.8771
2	4	1436.337	94.026	0.9204	-0.0704	0.8500
3	4	1436.337	40.134	0.9998	0.0017	1.0015
4	4	1436.337	56.351	0.9458	-0.1708	0.7750

Tabel 10: Estimated emission parameters for the AR(2) second-order HMM. $\hat{\sigma}$ and the AR(2) coefficients $\hat{\phi}_1, \hat{\phi}_2$ depend on (s_{t-1}, s_t) . A sum $\hat{\phi}_1 + \hat{\phi}_2 > 1$ indicates a non-stationary AR process for that state pair.

Table 10 reports the pair-specific emission parameters. The four state means are very close to the AR(1)-HMM(2) estimates, with a clear ordering from the baseline state 1 (400 ppm) up to the high-CO₂ state 4 (1436 ppm), and state 2 and 3 placed as intermediate states. As before, the standard deviations grow with the mean, which we attribute to more volatility in the CO₂ signal when occupants are present and windows are being opened and closed.

The first-order AR coefficients $\hat{\phi}_1$ remain strongly positive for most state pairs (typically 0.7–1.0), while the second-order coefficients $\hat{\phi}_2$ are small and mostly negative, ranging from -0.27 up to 0.17. This mirrors the pattern seen for the AR(2)-HMM. The observation at time t is dominated by the previous observation, while y_{t-2} contributes a small negative correction. The exception is the pair (1, 2), where $\hat{\phi}_1 = -0.28$ and $\hat{\phi}_2 = 0.17$, which corresponds to the system entering state 2 from the baseline and so the level is increasing.

Several pairs have $\hat{\phi}_1 + \hat{\phi}_2 > 1$, indicating a non-stationary AR process. These are the same kind of pairs that were flagged for the AR(1)-HMM(2). Transitions where the CO₂ level is changing sharply, such as (3, 4) and (4, 1), together with the self-transitions (3, 3) and (4, 3).

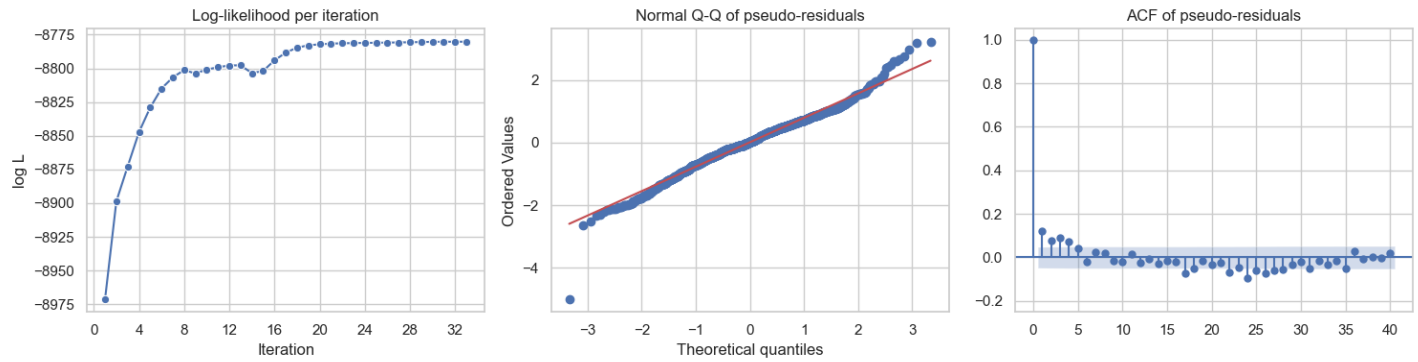


Figure 6: Diagnostic plots for the AR(2) second-order HMM. Left: log-likelihood per iteration. Centre: normal Q-Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

Figure 6 shows the diagnostics for the AR(2)-HMM(2). The convergence of the log-likelihood is not as smooth as previous models, but it still converges after a around 20 iterations. The Q-Q plot of the pseudo-residuals is similar to the previous models, with a few outliers in the tails. The ACF of the residuals shows a similar pattern as the previous models, with a slow decay and several lags outside the confidence band.

5 Model Comparison

We compare the five models in the hierarchy of Figure 1 using a likelihood ratio test (LRT) for each edge, as well as comparing the AIC and BIC.

Table 11 reports the per-model statistics for all five models in the hierarchy of Figure 1. Table 12 reports the LRT, p -value, Δ AIC and Δ BIC for each edge of the hierarchy, where the difference is taken as expanded minus base so that negative values favour the expanded model.

Model	#Params	LogLik	AIC	BIC
HMM(1)	20	-10388.87	20817.73	20926.10
AR(1), HMM(1)	24	-9359.73	18767.45	18897.47
AR(2), HMM(1)	28	-9273.19	18602.38	18754.06
AR(1), HMM(2)	84	-9130.90	18429.80	18884.88
AR(2), HMM(2)	100	-9045.40	18290.79	18832.49

Tabel 11: Per-model log-likelihood, AIC and BIC for the five fitted HMMs.

In the table, we see that the cost of increasing to second order markov chain introduces a lot of parameters, which is reflected in the AIC and BIC.

Base Model	Expanded Model	LRT	df	p -value	Δ AIC	Δ BIC
HMM(1)	AR(1), HMM(1)	2058.28	4	0.0000	-2050.28	-2028.61
AR(1), HMM(1)	AR(2), HMM(1)	173.07	4	0.0000	-165.07	-143.40
AR(1), HMM(1)	AR(1), HMM(2)	457.65	60	0.0000	-337.65	-12.60
AR(2), HMM(1)	AR(2), HMM(2)	455.59	72	0.0000	-311.59	78.43
AR(1), HMM(2)	AR(2), HMM(2)	171.01	16	0.0000	-139.01	-52.34

Tabel 12: Likelihood-ratio tests for each nested pair from the model hierarchy (Figure 1). Negative Δ AIC/ Δ BIC favour the expanded model.

We see that for almost all edges, all test statistics favour the more complex model, and the p -values are all 0. Only for the edge between the AR(2)-HMM(1) and the AR(2)-HMM(2) do the BIC favour the simpler model, while the LRT still favours the more complex model.

6 Conclusion

This project investigated whether room occupancy can be inferred from a single CO₂ sensor by modelling the observed series with a Hidden Markov Model. Five nested 4-state Gaussian HMMs were fitted, organised along the hierarchy of Figure 1. A baseline HMM(1), an AR(1) and AR(2) emission extension, a second-order latent chain with AR(1) emissions, and the fully combined AR(2), HMM(2) model. All models were implemented and fitted using `JAX`, `EQUINOX` and `Optax`.

The likelihood-ratio tests in Table 12 reject every base model in favour of its expanded counterpart at $p \approx 0$, confirming that both adding autoregressive structure to the emissions and making the Markov chain second-order produce statistically significant gains in fit. The single largest improvement is the introduction of an AR(1) emission on top of the ordinary HMM, with a likelihood-ratio statistic of $\Lambda \approx 2058$ on four degrees of freedom and a drop in AIC of more than two thousand units. This indicates that there indeed is a strong autocorrelation structure in the data that the plain HMM cannot capture it.

By looking at the ACF plots for all the models, we have seen that the lags are still significant for all models. This tells us, that there is still some temporal structure in the data we have not captured with the models we have implemented.

Overall, we have seen that extending the ordinary with higher order markov chains or autoregressive emissions gives a significant improvement in fit, and tells us that there is indeed temporal structure in the data. However, there is still some temporal structure in the data that we have not captured with the models we have implemented, so there is still room for improvement.

7 Future Work

Ideas for future work:

- Optimizing the code to find optimal parameters faster. Right now it takes a bit of time to fit the models.
- Extend the models to capture more temporal structure.
- Predicting the occupancy state from the CO₂ measurements, and evaluating the predictive performance of the models.
- Testing and implementing different optimization algorithms and methods for estimating uncertainty of the parameters.

Litteratur

- Astral Software Inc. uv: An ultra-fast python package installer and resolver. <https://docs.astral.sh/uv/>, 2024. Accessed: 2026-05-30.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- Matteo Hessel, David Budden, Fabio Viola, Mihaela Rosca, Eren Sezener, and Tom Hennigan. Optax: composable gradient transformation and optimisation, in jax!, 2020. URL <http://github.com/deepmind/optax>.
- Patrick Kidger and Cristian Garcia. Equinox: neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming workshop at Neural Information Processing Systems 2021*, 2021.
- Jan Kloppenborg Møller, Marcel Schweiker, Rune Korsholm Andersen, Burak Gunay, Selin Yilmaz, Verena Marie Barthelmes, and Henrik Madsen. *Statistical Modelling of Occupant Behaviour*. A Chapman & Hall Book. Chapman and Hall/CRC, 1st edition, 2024.
- Ultralytics LLC. Hidden markov model (hmm) - real-world applications. <https://www.ultralytics.com/glossary/hidden-markov-model-hmm#real-world-applications>, 2024. Accessed: 2026-05-30.
- Wikipedia contributors. Akaike information criterion — Wikipedia, the free encyclopedia, 2026a. URL https://en.wikipedia.org/wiki/Akaike_information_criterion. [Online; accessed 31-May-2026].
- Wikipedia contributors. Bayesian information criterion — Wikipedia, the free encyclopedia, 2026b. URL https://en.wikipedia.org/wiki/Bayesian_information_criterion. [Online; accessed 31-May-2026].
- Wikipedia contributors. Likelihood-ratio test — Wikipedia, the free encyclopedia, 2026c. URL https://en.wikipedia.org/wiki/Likelihood-ratio_test. [Online; accessed 31-May-2026].
- Walter Zucchini, Iain L. MacDonald, and Roland Langrock. *Hidden Markov Models for Time Series: An Introduction Using R*. Number 150 in Monographs on Statistics and Applied Probability. CRC Press, second edition, 2016.

8 Appendix

8.1 Source Code Repository

The repository is hosted on GitHub and the source code can be found there with the data as well: <https://github.com/Maccen26/hmm-project>

8.2 Source code & Envoriment

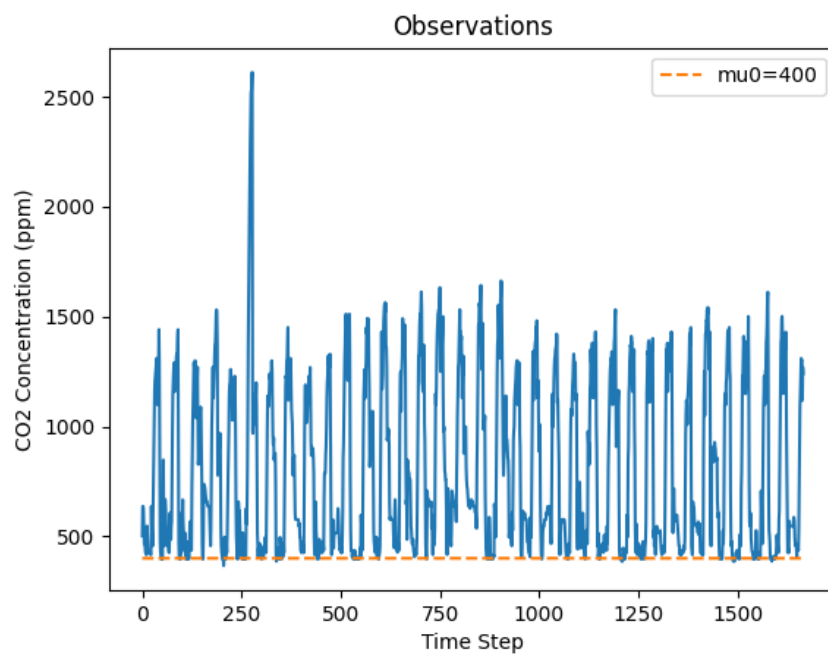
This code was written with Python (version 3.11) and uv (version 0.9.7) (Astral Software Inc., 2024) was used as a package manager. The libraries used are listed in Table 13.

Library	Version
dotenv	$\geq 0.9.9$
equinox	$\geq 0.13.7$
jax	$\geq 0.9.0.1$
matplotlib	$\geq 3.10.8$
numpy	$\geq 1.23.5, < 2.3$
optax	$\geq 0.2.8$
pandas	$\geq 3.0.0$
seaborn	$\geq 0.13.2$
statsmodels	$\geq 0.14.6$

Tabel 13: Python libraries used in this project.

The source code is organized in a way such that the directory **src** contains the source code for the project, the directory **data** contains the data used for training and testing the model, the directory **tests** contains tests for the source code and the directory **drivers** contains scripts for running the model and generating the results used in the report.

8.3 CO₂ Data



Figur 7: Observed CO₂ concentration (ppm) over time.

8.4 Class Diagrams

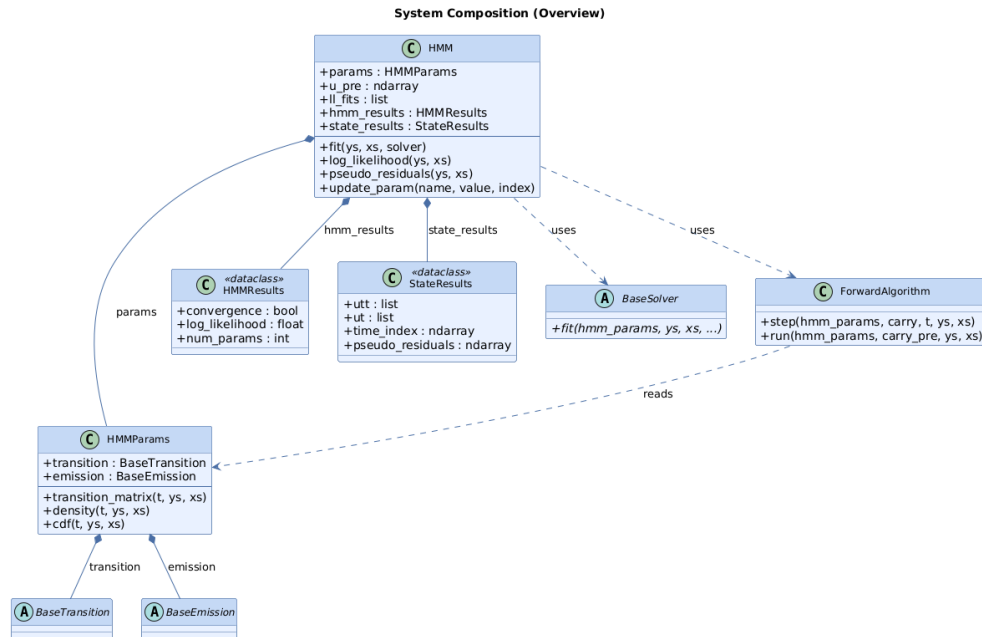


Figure 8: System composition overview. The user-facing **HMM** class owns an **HMMParams** instance composed of a **BaseTransition** and a **BaseEmission**. Fitting is delegated to a **BaseSolver** subclass and state inference is handled by **ForwardAlgorithm**, producing **HMMResults** and **StateResults**.

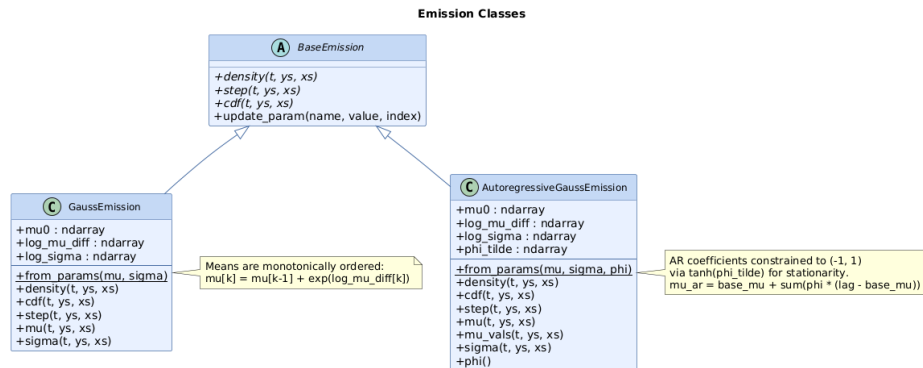


Figure 9: Emission class hierarchy. **GaussEmission** models state-dependent Gaussian distributions with monotonically ordered means enforced via cumulative exponentiation of **log_mu_diff**. **AutoregressiveGaussEmission** extends this with $AR(p)$ dynamics, where coefficients are stored unconstrained (**phi_tilde**) and mapped to $(-1, 1)$ via $\tanh$ to guarantee stationarity.

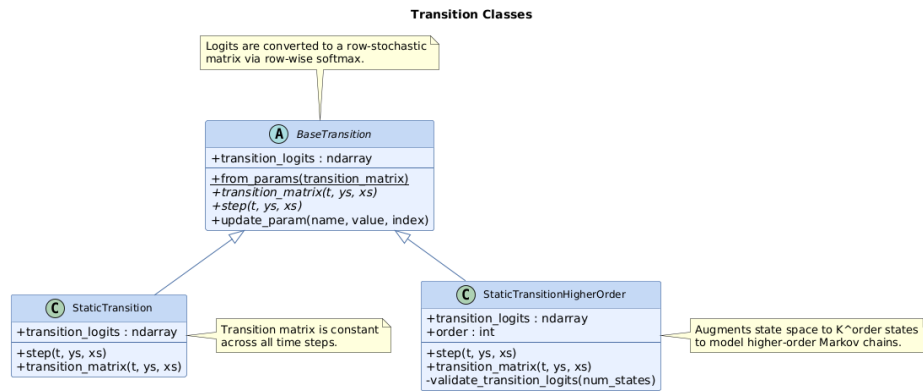


Figure 10: Transition class hierarchy. Both implementations store parameters as logits and convert them to a row-stochastic matrix via row-wise softmax. `StaticTransition` uses a constant $K \times K$ matrix, while `StaticTransitionHigherOrder` augments the state space to K^{order} states to represent a higher-order Markov chain.

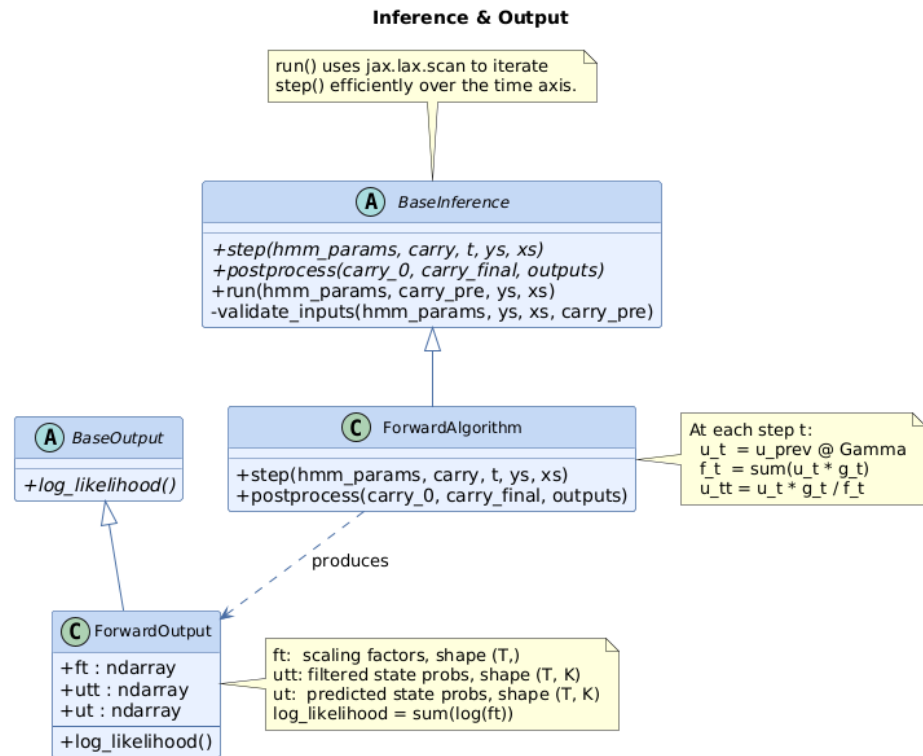


Figure 11: Inference and output class hierarchy. `BaseInference.run()` iterates `step()` across the time axis using `jax.lax.scan`. `ForwardAlgorithm` implements the forward filtering pass and produces a `ForwardOutput` storing the scaling factors f_t , filtered state probabilities $u_{t|t}$, and predicted state probabilities u_t . The log-likelihood is computed as $\sum_t \log f_t$.

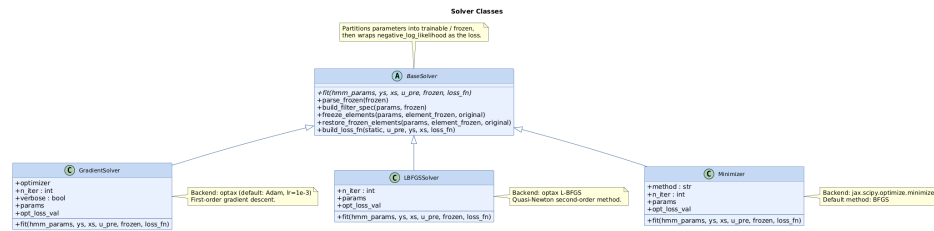


Figure 12: Solver class hierarchy. **BaseSolver** partitions parameters into trainable and frozen subsets and wraps the negative log-likelihood as the loss function. The three concrete solvers share the same interface but differ in their optimization backend: **GradientSolver** uses first-order gradient descent via **optax**, **LBFGSSolver** uses quasi-Newton L-BFGS via **optax**, and **Minimizer** wraps **jax.scipy.optimize.minimize**.

8.5 Class Design

The **HMM** class (8) is the main entry point of the system. Its responsibility is to act as a wrapper for the other components and to provide a simple interface for training and inference. It is dependent on **Emissions**, **Transitions**, **HMMParams**, and **Solvers**, which are composed together to form a complete HMM.

The **Emissions** (9) and **Transitions** (10) classes are abstract interfaces whose concrete implementations must be provided at runtime. This design choice is motivated by the structure of HMM variants: the differences between models in this project lie in how the density of an observation given a state is computed (emissions) and how the transition probabilities between states are computed (transitions). By separating these concerns into abstract interfaces, different HMM models can be composed without changing the rest of the system.

The **HMMParams** class acts as a container for the model parameters and as a wrapper for the methods implemented in the **Emissions** and **Transitions** classes. This is required by how **JAX** and **Equinox** work: parameters must be stored as pytrees to support automatic differentiation.

The **Inference** class (11) is an abstract interface responsible for implementing inference algorithms such as the forward algorithm, the backward algorithm, and the Viterbi algorithm. In this project, only the forward algorithm is implemented, but the design allows easy extension to other inference algorithms without modifying existing code.

The **Solvers** class (12) implements the optimization algorithms and depends on a loss function that takes the **HMMParams** and **Inference** classes as arguments. This dependency injection of the loss function decouples the optimizer from the objective, making it straightforward to change the loss function without altering the optimization algorithm.